Лабораторная работа №4. Работа с файлово-каталожной системой в ОС Linux

Рассматриваемые вопросы

1. Основные команды для работы с файлами и каталогами.
2. Использование механизма ссылок.
3. Прямая и косвенная адресация каталогов.

Основные команды для работы с файлами и каталогами

- `cd` - смена текущего каталога.
- `cp` - копирование файлов и каталогов.
- `ls` - вывод списка файлов и каталогов в указанном каталоге.
- `file` - определение типа указанного файла.
- `find` - поиск файлов и каталогов по заданным критериям.
- `ln` - создание жёстких и символьных ссылок.
- `mkdir` - создание каталога.
- `mv` - перемещение (переименование) файла или каталога.
- `pwd` - вывод абсолютного пути текущего каталога.
- `rm` - удаление файлов (и при необходимости каталогов с параметрами).
- `rmdir` - удаление пустых каталогов.
- `cat` - вывод содержимого файлов и/или конкатенация файлов.

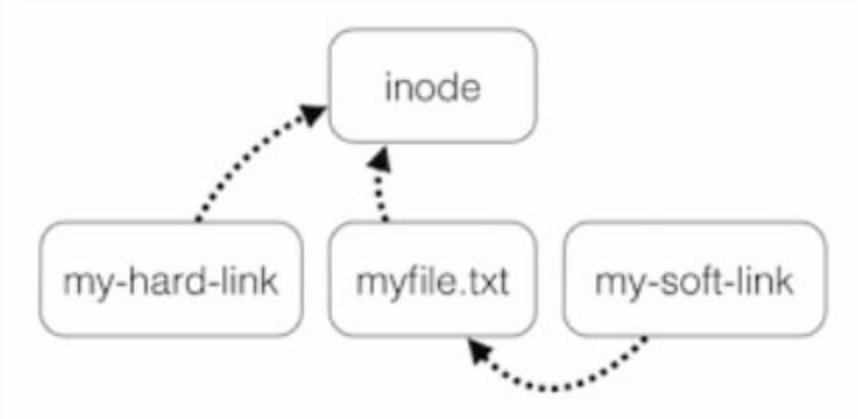
Ссылки на файлы

В Linux используются два вида ссылок: **жёсткие** и **символьные** (мягкие).

Жёсткая ссылка - это запись в каталоге, связывающая имя файла с тем же inode, что и исходный файл. Все жёсткие ссылки на один и тот же inode равноправны и указывают на одни и те же данные. Следовательно, один файл может иметь несколько имён, в том числе в разных каталогах. Физическое удаление данных с диска происходит только после удаления **последней** жёсткой ссылки (т. е. когда счётчик ссылок inode становится равным нулю). Понятия «настоящее имя файла» в этом случае не существует: все имена эквивалентны.

Символьная ссылка (*symlink*) принципиально отличается от жёсткой: это отдельный специальный файл со своим inode, который хранит путь к целевому файлу/каталогу.

Символьная ссылка может указывать на объекты в других файловых системах и может стать недействительной, если целевой объект будет перемещён или удалён.



При обращении к символьной ссылке ядро ОС интерпретирует содержащийся в ней путь и выполняет операцию над целевым объектом. При этом команда `rm` удаляет **саму ссылку**, а не файл/каталог, на который она указывает. Для получения значения символьной ссылки (целевого пути) используется команда `readlink`.

Полное имя файла может задаваться **абсолютным** путём (например, `/home/user/file`) или **относительным** путём - относительно текущего каталога (часто используется в скриптах).

В каждом каталоге присутствуют служебные записи:

- `..` - родительский каталог;
- `.` - текущий каталог.

Примеры:

- `cd ..` - переход на уровень выше;
- `cd .` - переход в текущий каталог (фактически ничего не изменяет);
- `./script.bash` - запуск скрипта из текущего каталога.

Для перехода в корень файловой системы используется `cd /`. Домашний каталог текущего пользователя может задаваться через `~`, поэтому `cd ~` эквивалентна `cd "$HOME"`.

Задание на лабораторную работу

Разработайте скрипты для перечисленных заданий и продемонстрируйте их преподавателю.

Внимание. Все скрипты должны корректно обрабатывать различные сценарии использования, включая (но не ограничиваясь): некорректный ввод параметров, имена

файлов с «необычными», но допустимыми символами, а также различные последовательности запусков скриптов и действий пользователя в файловой системе. Все ошибки (в том числе ошибки системных утилит) должны быть обработаны; пользователю необходимо выводить содержательные диагностические сообщения.

1. Механизм ссылок

- Создайте следующую структуру каталогов:

```
some_project/
| - versions/
  | - v1/
    | - README.md
  | - v2/
    | - README.md
  | - v3/
    | - README.md
| - current_version
```

- Создайте символьную ссылку `current_version -> versions/v1`.
- Создайте каталог внутри `some_project/`. Внутри него создайте жёсткие ссылки на все файлы из каталога, на который указывает `current_version` (требование: выполнить одной командой).
- Измените ссылку `current_version -> versions/v2` и объясните, что изменилось и почему.

2. Система контроля версий

Разработайте скрипт, реализующий систему контроля версий **одного** файла и использующий механизм жёстких ссылок для хранения версий. Изменения файла определяйте по изменению его inode.

- В домашнем каталоге создайте скрытый каталог для хранения данных системы версионирования.

`fileversion init <путь_к_файлу>`

Действия:

1. Проверить существование файла. Если файл отсутствует - завершить работу с ошибкой.
2. Определить базовое имя файла (без пути).

3. Создать каталог `~/<скрытая_директория_для_хранения_версий>/<имя_отслеживаемого_файла>/`.
4. Создать подкаталог `versions/`.
5. Создать первую жёсткую ссылку на текущий inode файла в `versions/` с именем формата `v1_YYYY-MM-DD_HH-MM-SS`.
6. Создать символьную ссылку `current_version`, указывающую на первую версию.
7. Создать файл `metadata.json` со следующей структурой:

```
{
  "filename": "<полный_путь_к_исходному_файлу>",
  "basename": "<базовое_имя_файла>",
  "versions": [
    {
      "id": 1,
      "name": "v1_YYYY-MM-DD_HH-MM-SS",
      "timestamp": "YYYY-MM-DD HH:MM:SS",
      "inode": 1234567,
      "size": 1024,
      "comment": "Initial version",
      "hash": "sha256:<хэш_содержимого>"
    }
  ],
  "current_version": 1,
  "created": "YYYY-MM-DD HH:MM:SS",
  "last_updated": "YYYY-MM-DD HH:MM:SS"
```

```
}
```

8. Вывести сообщение: `Version control initialized for <путь_к_файлу>`.

`fileversion commit <путь_к_файлу> ["комментарий"]`

Действия:

1. Проверить, инициализирована ли система контроля версий для файла (существует ли `~/<скрытая_директория_для_хранения_версий>/<базовое_имя_файла>/`).
2. Определить inode текущего файла.
3. Определить inode версии, на которую указывает `current_version`.
4. Выполнить сравнение inode:

- если `current_inode == stored_inode`, то файл не изменился; вывести `No changes` и завершить работу;

- если `current_inode != stored_inode`, то файл изменён и требуется фиксация версии.

5. Проверить, существует ли уже версия с таким inode (защита от дублирования).
6. Если версия найдена - использовать существующую ссылку, не создавая новую.
7. Создать новую жёсткую ссылку в `versions/` с именем `v<N>_YYYY-MM-DD_HH-MM-SS`, где `N` - следующий порядковый номер.
8. Обновить символьную ссылку `current_version`, направив её на новую версию.
9. Обновить `metadata.json`:

- добавить запись о новой версии в массив `versions`;

- обновить `current_version`;

- обновить `last_updated`;

- вычислить SHA-256 хэш содержимого файла и сохранить в метаданных.

10. Вывести сообщение `Committed version N: <комментарий>`.

`fileversion restore <путь_к_файлу> <version>`

Требование: в аргументе `<version>` поддержать значение `latest` (последняя версия).

Действия:

1. Проверить существование указанной версии в `metadata.json`.

2. Определить путь к файлу версии: `~/<скрытая_директория_для_хранения_версий>/<имя>/versions/<имя_версии>`.
3. **Важно:** восстановление должно создавать новую жёсткую ссылку на существующий inode, а не копировать содержимое файла.
4. Выполнить восстановление:

- удалить текущий файл по указанному пути;

- создать жёсткую ссылку из версии на исходный путь.

5. Обновить `current_version` в системе версий на восстановленную версию.
6. Обновить `metadata.json` (новое значение `current_version` и время изменения, если предусмотрено).
7. Вывести сообщение: `Checked out version X to <путь>`.